

PANDAS

pandas is a python library it is used for data analysis and data manipulation

why it is used

pandas is used for storing,reading,cleaning, and analysing the data

installing pandas

```
In [1]: # pip install pandas
```

importing pandas

```
In [2]: import pandas as pd
```

Data structures in pandas

pandas provide two datastructures for manipulating the data

1) Pandas Series

A pandas series is a one dimensional labelled array it is capable of holding any data type(int,float,string)

creating a pandas series

```
In [3]: s=pd.Series([1,3,5,7,9])  
print(s)
```

```
0    1  
1    3  
2    5  
3    7  
4    9  
dtype: int64
```

```
In [4]: s=pd.Series(["Akshitha",19,50.5,21])  
print(s)
```

```
0    Akshitha  
1         19  
2        50.5  
3         21  
dtype: object
```

```
In [5]: import pandas as pd  
import numpy as np
```

```
ser=pd.Series()
print("pandas Series",ser)
data=np.array(['p','y','t','h','o','n'])
ser=pd.Series(data)
print("Pandas Series:\n",ser)
```

```
pandas Series Series([], dtype: object)
Pandas Series:
 0    p
 1    y
 2    t
 3    h
 4    o
 5    n
dtype: object
```

2)Pandas data frame

pandas dataframe is a two dimensional data structure it is used to store data in rows and columns just like an excel

```
In [6]: import pandas as pd
df=pd.DataFrame()
print(df)
lst=["python", "is", "powerful", "language"]
df=pd.DataFrame(lst)
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
 0
0  python
1      is
2  powerful
3  language
```

creating an empty pandas series

```
In [7]: ser=pd.Series()
print(ser)
```

```
Series([], dtype: object)
```

creating a series from numpy array

```
In [8]: import pandas as pd
import numpy as np
data=np.array(['p','y','t','h','o','n'])
ser=pd.Series(data)
print(ser)
```

```
0    p
1    y
2    t
3    h
4    o
5    n
dtype: object
```

creating a series from list

```
In [9]: data_list=["data science",2018,13.5,"easy"]
        ser1=pd.Series(data_list)
        print(ser1)
```

```
0    data science
1           2018
2           13.5
3           easy
dtype: object
```

creating a series from a dectionary

```
In [10]: data_dict={"name":"india","age":25,'eligible':"yes"}
         ser2=pd.Series(data_dict)
         print(ser2)
```

```
name      india
age        25
eligible   yes
dtype: object
```

common methods of series

`s.describe()`:- describe function is used to provide summary statistics of the data

```
In [11]: s=pd.Series([1,3,5,7,9])
         print(s)
```

```
0    1
1    3
2    5
3    7
4    9
dtype: int64
```

```
In [12]: s.describe()
```

```
Out[12]: count    5.000000
         mean     5.000000
         std      3.162278
         min      1.000000
         25%      3.000000
         50%      5.000000
         75%      7.000000
         max      9.000000
         dtype: float64
```

mean():- calculates the mean of the data

```
In [13]: print(s.mean())
```

5.0

min() :- calculates the minimum value in a series

max():- calculates the maximum value in a series

```
In [14]: print(s.min())
         print(s.max())
```

1

9

```
In [15]: data=np.array(['p','y','t','h','o','n'])
         ser=pd.Series(data)
         print(ser)
```

0 p

1 y

2 t

3 h

4 o

5 n

dtype: object

```
In [16]: print(ser[0])
```

p

```
In [17]: print(ser[0:4])
```

0 p

1 y

2 t

3 h

dtype: object

```
In [18]: print(ser[:])
```



```

0    p
1    y
2    t
3    h
4    o
5    n
dtype: object

```

```
In [19]: print(ser[::-1])
```

```

5    n
4    o
3    h
2    t
1    y
0    p
dtype: object

```

Accessing a Single Element Using index Label

```
In [20]: data=np.array(['p','y','t','h','o','n'])
ser=pd.Series(data,index=[11,12,13,14,15,16])
print(ser[14])
```

```
h
```

Accessing a Multiple Element Using index Label

```
In [21]: data=np.array(['p','y','t','h','o','n'])
ser=pd.Series(data,index=[11,12,13,14,15,16])
print(ser[[14,11,12]])
```

```

14    h
11    p
12    y
dtype: object

```

```
In [22]: ser=pd.Series(np.arange(3,9),index=['a','b','c','d','e','f'])
print(ser[['a','d']])
```

```

a     3
d     6
dtype: int64

```

Binary Operations on Pandas Series

```
In [23]: s1=pd.Series([10,20,30],index=['a','b','c'])
s2=pd.Series([1,2,3],index=['a','b','c'])
result=s1+s2
print(result)
```

```

a     11
b     22
c     33
dtype: int64

```

```
In [24]: s1=pd.Series([10,20,30])
s2=pd.Series([10,25,30])
result=s1==s2
print(result)
```

```
0    True
1    False
2     True
dtype: bool
```

Binary Operations on Pandas DataFrame

```
In [25]: df1=pd.DataFrame({'A': [10,20,30], 'B': [40,50,60]})
df2=pd.DataFrame({'A': [1,2,3], 'B': [4,5,6]})
result=df1-df2
print(result)
```

```
   A  B
0  9 36
1 18 45
2 27 54
```

```
In [26]: df1=pd.DataFrame({'A': [10,20,30], 'B': [40,50,60]})
df2=pd.DataFrame({'A': [1,2,3], 'B': [4,5,6]})
result=df1>df2
print(result)
```

```
   A  B
0 True True
1 True True
2 True True
```

```
In [27]: s1=pd.Series([True,False,True])
s2=pd.Series([False,False,True])
result=s1 & s2
print(result)
```

```
0    False
1    False
2     True
dtype: bool
```

Data manipulation

Data manipulation is the process of cleaning ,organising and transforming data

`s.map()` :- applies a function to each and every element in the series it is only used in series

```
In [28]: s=pd.Series([1,5,3,12,7])
doubled=s.map(lambda x:x**2)
print(doubled)
```

```

0      1
1     25
2      9
3    144
4     49
dtype: int64

```

lambda function is anonymous function in pandas that can create without def

s.apply():- it is same as the map function it is also used in dataframe

```

In [29]: sqrt=s.apply(lambda x: x**0.5)
         print(sqrt)

```

```

0      1.000000
1      2.236068
2      1.732051
3      3.464102
4      2.645751
dtype: float64

```

s.sort_values():-sort the series

```

In [30]: s=pd.Series([4,1,0,99,52])
         sorted_s=s.sort_values()
         print(sorted_s)

```

```

2      0
1      1
0      4
4     52
3     99
dtype: int64

```

s.drop(labels):-drop specific label from the series

```

In [31]: dropped=s.drop(0)
         print(dropped)

```

```

1      1
2      0
3     99
4     52
dtype: int64

```

```

In [32]: s

```

```

Out[32]: 0      4
         1      1
         2      0
         3     99
         4     52
         dtype: int64

```

```

In [33]: dropped=s.drop(0,inplace=True)
         print(dropped)

```

None

In [34]:

s

Out[34]:

```
1    1
2    0
3    99
4    52
dtype: int64
```

Handling missing data

`s.isnull()` : checks for missing values, returns a series of boolean¶

In [35]:

```
# checking for missing values
print(s.isnull())
```

```
1    False
2    False
3    False
4    False
dtype: bool
```

`s.notnull()` : opposite of `isnull()`¶

In [36]:

```
# checking for missing values

print(s.notnull())
```

```
1    True
2    True
3    True
4    True
dtype: bool
```

In [37]:

```
# create a series with missing values
# nan-not a number

s=pd.Series([1,2,np.nan,4,np.nan])

#print the series
print(s)
```

```
0    1.0
1    2.0
2    NaN
3    4.0
4    NaN
dtype: float64
```

In [38]:

```
s.isnull()
```

```
Out[38]: 0    False
         1    False
         2     True
         3    False
         4     True
         dtype: bool
```

```
In [39]: s.isnull().sum()
```

```
Out[39]: np.int64(2)
```

s.isnull().sum()

```
In [40]: # filling missing values with 0
```

```
filled=s.fillna(0)
print(filled)
```

```
0    1.0
1    2.0
2    0.0
3    4.0
4    0.0
dtype: float64
```

s.dropna(): Drops all rows that contain missing values

```
In [41]: array=pd.Series([1,2,None,4,5])
         dropped_missing=array.dropna()
         print(dropped_missing)
```

```
0    1.0
1    2.0
3    4.0
4    5.0
dtype: float64
```

```
In [42]: array
```

```
Out[42]: 0    1.0
         1    2.0
         2    NaN
         3    4.0
         4    5.0
         dtype: float64
```

loc-Label based indexing:- you use row/column label

iloc-integer location based indexing-you use row/column positions

```
In [43]: s=pd.Series([1,3,5,7,9])
         print(s.iloc[0])
         print(s.iloc[4])
```

```
1
9
```

```
In [44]: print(s.loc[0])  
         print(s.loc[4])
```

```
1  
9
```

```
In [45]: s=pd.Series([10,20,30,40], index=['a','b','c','d'])  
         print(s)  
         print(s.iloc[0])  
         print(s.iloc[1:3])  
         print(s.loc['a'])  
         print(s.loc['b':'c'])
```

```
a    10  
b    20  
c    30  
d    40  
dtype: int64  
10  
b    20  
c    30  
dtype: int64  
10  
b    20  
c    30  
dtype: int64
```

```
In [46]: filtered=s[s>20]  
         print(filtered)
```

```
c    30  
d    40  
dtype: int64
```

Data Frame

Read the csv file

```
In [47]: df=pd.read_csv(r"C:\Users\akshi\Downloads\Movie-Rating.csv")
```

```
In [48]: df
```

Out[48]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

In [49]: `df.head()`

Out[49]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [50]: `df.tail()`

Out[50]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

Summary of Data Frame

In [51]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Film                                  559 non-null    object
1   Genre                                559 non-null    object
2   Rotten Tomatoes Ratings %           559 non-null    int64
3   Audience Ratings %                  559 non-null    int64
4   Budget (million $)                  559 non-null    int64
5   Year of release                      559 non-null    int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

Missing data handling

Find missing values in the dataset

In [52]: df.isnull()

Out[52]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
554	False	False	False	False	False	False
555	False	False	False	False	False	False
556	False	False	False	False	False	False
557	False	False	False	False	False	False
558	False	False	False	False	False	False

559 rows × 6 columns

Find the number of missing values in the dataset

In [53]: df.isnull().sum()

Out[53]: Film 0
Genre 0
Rotten Tomatoes Ratings % 0
Audience Ratings % 0
Budget (million \$) 0
Year of release 0
dtype: int64

Removing missing values

df.dropna():- function removes columns or rows which contains atleast one missing values

In [54]: df.dropna()

Out[54]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

`data_frame.dropna(axis = 1)` drops the columns where at least one element is missing.

Fill in missing values

`df.fillna()`:-function is used to fill the missing values

In [55]: `df.fillna(0)`

Out[55]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

Removing rows

df.drop(index)

In [56]: df.describe()

	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
count	559.000000	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136	2009.152057
std	26.413091	16.826887	48.731817	1.362632
min	0.000000	0.000000	0.000000	2007.000000
25%	25.000000	47.000000	20.000000	2008.000000
50%	46.000000	58.000000	35.000000	2009.000000
75%	70.000000	72.000000	65.000000	2010.000000
max	97.000000	96.000000	300.000000	2011.000000

In [57]: df.shape

Out[57]: (559, 6)

```
In [58]: df.columns
```

Out[58]: Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %',
 'Budget (million \$)', 'Year of release'],
 dtype='object')

```
In [59]: df.dtypes
```

Out[59]: Film object
Genre object
Rotten Tomatoes Ratings % int64
Audience Ratings % int64
Budget (million \$) int64
Year of release int64
dtype: object

```
In [60]: df.columns=["Film","Genre","CriticRating","AudienceRatings ","BudgetMillion","Year"]
```

```
In [61]: df
```

Out[61]:

	Film	Genre	CriticRating	AudienceRatings	BudgetMillion	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

```
In [62]: df=df.rename(columns={"BudgetMillion":"Budget"})
```

```
In [63]: df.head()
```

Out[63]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [64]: `df["Film"]`

```
Out[64]: 0      (500) Days of Summer
1      10,000 B.C.
2      12 Rounds
3      127 Hours
4      17 Again
...
554     Your Highness
555     Youth in Revolt
556     Zodiac
557     Zombieland
558     Zookeeper
Name: Film, Length: 559, dtype: object
```

In [65]: `titles=df.loc[:,"Film"]`
`titles`

```
Out[65]: 0      (500) Days of Summer
1      10,000 B.C.
2      12 Rounds
3      127 Hours
4      17 Again
...
554     Your Highness
555     Youth in Revolt
556     Zodiac
557     Zombieland
558     Zookeeper
Name: Film, Length: 559, dtype: object
```

In [66]: *# selecting a range of rows and multiple columns by labels*

```
subset=df.loc[10:20,["Film","Genre","Year"]]
subset
```

Out[66]:

	Film	Genre	Year
10	88 Minutes	Drama	2007
11	A Dangerous Method	Drama	2011
12	A Nightmare on Elm Street	Horror	2010
13	A Serious Man	Drama	2009
14	A Very Harold and Kumar Christmas	Comedy	2011
15	Abduction	Action	2011
16	Across the Universe	Romance	2007
17	Adventureland	Comedy	2009
18	Albert Nobbs	Drama	2011
19	Alice in Wonderland	Adventure	2010
20	Alien vs. Predator -- Requiem	Horror	2007

```
In [67]: type_of_movie=df[df["Genre"]=="Comedy"]
         type_of_movie
```

Out[67]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
4	17 Again	Comedy	55	70	20	2009
6	27 Dresses	Comedy	40	71	30	2008
8	30 Minutes or Less	Comedy	43	48	28	2011
9	50/50	Comedy	93	93	8	2011
...	...	...	...	...	...	...
552	You Don't Mess with the Zohan	Comedy	36	50	80	2008
553	You Will Meet a Tall Dark Stranger	Comedy	43	35	22	2010
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
558	Zookeeper	Comedy	14	42	80	2011

172 rows × 6 columns

```
In [68]: df[df["Genre"]=="Comedy"]
```

Out[68]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
4	17 Again	Comedy	55	70	20	2009
6	27 Dresses	Comedy	40	71	30	2008
8	30 Minutes or Less	Comedy	43	48	28	2011
9	50/50	Comedy	93	93	8	2011
...	...	...	...	...	...	...
552	You Don't Mess with the Zohan	Comedy	36	50	80	2008
553	You Will Meet a Tall Dark Stranger	Comedy	43	35	22	2010
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
558	Zookeeper	Comedy	14	42	80	2011

172 rows × 6 columns

```
In [69]: df["CriticRating"].max()
```

Out[69]: 97

```
In [70]: sep= df[(df["Year"]==2008) & (df["Genre"]=="Comedy")]
sep
```

Out[70]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
6	27 Dresses	Comedy	40	71	30	2008
35	Baby Mama	Comedy	63	59	70	2008
41	Be Kind, Rewind	Comedy	65	57	20	2008
43	Bedtime Stories	Comedy	25	63	80	2008
57	Burn After Reading	Comedy	78	64	37	2008
87	Definitely, Maybe	Comedy	71	72	7	2008
93	Disaster Movie	Comedy	2	28	20	2008
104	Drillbit Taylor	Comedy	26	49	40	2008
129	First Sunday	Comedy	13	59	13	2008
133	Forgetting Sarah Marshall	Comedy	85	75	30	2008
134	Four Christmases	Comedy	26	52	80	2008
147	Ghost Town	Comedy	85	60	20	2008
172	High School Musical 3: Senior Year	Comedy	65	76	11	2008
220	Leatherheads	Comedy	52	41	58	2008
234	Mad Money	Comedy	22	51	22	2008
235	Made of Honor	Comedy	13	61	40	2008
236	Mamma Mia!	Comedy	53	76	66	2008
238	Marley and Me	Comedy	63	77	55	2008
242	Meet the Spartans	Comedy	2	31	30	2008
247	Miss Pettigrew Lives for a Day	Comedy	78	70	60	2008
256	My Best Friend's Girl	Comedy	14	51	20	2008
269	Nick and Norah's Infinite Playlist	Comedy	73	67	10	2008
289	Over Her Dead Body	Comedy	15	47	10	2008
298	Penelope	Comedy	52	74	15	2008
328	Role Models	Comedy	77	75	28	2008
337	Semi-Pro	Comedy	21	41	3	2008
339	Sex and the City	Comedy	49	81	58	2008
352	Smart People	Comedy	50	41	7	2008
356	Soul Men	Comedy	45	42	0	2008

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
364	Step Brothers	Comedy	54	68	65	2008
372	Sunshine Cleaning	Comedy	72	59	5	2008
375	Superhero Movie	Comedy	15	36	65	2008
377	Swing Vote	Comedy	36	44	20	2008
430	The House Bunny	Comedy	39	54	25	2008
447	The Longshots	Comedy	39	39	30	2008
449	The Love Guru	Comedy	14	38	62	2008
505	Tropic Thunder	Comedy	83	69	90	2008
535	Welcome Home, Roscoe Jenkins	Comedy	24	64	35	2008
536	What Happens in Vegas	Comedy	28	72	35	2008
549	Yes Man	Comedy	43	72	70	2008
552	You Don't Mess with the Zohan	Comedy	36	50	80	2008

```
In [71]: sep.shape
```

Out[71]: (41, 6)

```
In [72]: single_row=df.iloc[0]
single_row
```

Out[72]: Film (500) Days of Summer
Genre Comedy
CriticRating 87
AudienceRatings 81
Budget 8
Year 2009
Name: 0, dtype: object

```
In [73]: df.head(1)
```

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009

```
In [74]: specific_data=df.iloc[10,[1,2,3]]
specific_data
```

Out[74]: Genre Drama
CriticRating 5
AudienceRatings 51
Name: 10, dtype: object

```
In [75]: multi_slice=df.iloc[10:15,0:4]
multi_slice
```

Out[75]:

	Film	Genre	CriticRating	AudienceRatings
10	88 Minutes	Drama	5	51
11	A Dangerous Method	Drama	79	89
12	A Nightmare on Elm Street	Horror	13	40
13	A Serious Man	Drama	89	64
14	A Very Harold and Kumar Christmas	Comedy	72	71

df.at:- is used to access a single value in the dataframe using the row label and column label

```
In [76]: budget=df.at[0, 'Budget']
budget
```

Out[76]: np.int64(8)

```
In [77]: year=df[df['Year']>2008]
```

```
In [78]: year
```

Out[78]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
5	2012	Action	39	63	200	2009
...	...	...	...	...	...	...
553	You Will Meet a Tall Dark Stranger	Comedy	43	35	22	2010
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

358 rows × 6 columns

```
In [79]: df.head()
```

Out[79]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

```
In [80]: film_year=df[(df['AudienceRatings ']>75) & (df['Year']<2010)]  
film_year
```

Out[80]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
16	Across the Universe	Romance	54	84	45	2007
25	American Gangster	Thriller	79	87	100	2007
26	An Education	Drama	94	78	8	2009
33	Avatar	Action	83	92	237	2009
62	Changeling	Drama	62	84	55	2008
64	City Island	Comedy	81	77	6	2009
94	District 9	Action	91	81	30	2009
113	Enchanted	Comedy	93	80	85	2007
123	Fast and Furious	Action	28	76	85	2009
151	Gran Torino	Drama	80	90	33	2008
155	Grindhouse	Action	82	86	53	2007
158	Hachiko: A Dog's Story	Drama	58	85	16	2009
168	Harry Potter and the Order of the Phoenix	Adventure	78	82	150	2007
172	High School Musical 3: Senior Year	Comedy	65	76	11	2008
192	Inglourious Basterds	Adventure	88	87	70	2009
195	Into the Wild	Drama	82	90	15	2007
196	Iron Man	Action	94	91	186	2008
207	Juno	Comedy	94	89	8	2007
215	Knocked Up	Comedy	91	83	33	2007
228	Live Free or Die Hard	Action	79	86	110	2007
236	Mamma Mia!	Comedy	53	76	66	2008
238	Marley and Me	Comedy	63	77	55	2008
275	No Country for Old Men	Drama	95	84	25	2007
290	P.S. I Love You	Romance	21	82	30	2007
338	Seven Pounds	Drama	27	77	55	2008
339	Sex and the City	Comedy	49	81	58	2008
345	Sherlock Holmes	Action	70	81	90	2009
348	Shooter	Action	47	82	61	2007

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
362	Star Trek	Action	94	91	140	2009
363	Stardust	Adventure	76	86	70	2007
365	Step Up 2: The Streets	Drama	26	81	18	2008
374	Superbad	Comedy	88	87	20	2007
380	Taken	Action	58	83	25	2008
394	The Bourne Ultimatum	Thriller	93	91	110	2007
396	The Boy in the Striped Pyjamas	Drama	63	84	13	2008
398	The Bucket List	Drama	41	81	45	2007
400	The Chronicles of Narnia: Prince Caspian	Action	66	78	200	2008
405	The Darjeeling Limited	Drama	67	79	18	2007
406	The Dark Knight	Thriller	94	96	185	2008
422	The Hangover	Comedy	79	87	35	2009
431	The Hurt Locker	Drama	97	83	15	2008
441	The Kingdom	Action	52	78	80	2007
442	The Kite Runner	Drama	66	85	20	2007
467	The Secret Life of Bees	Drama	56	78	11	2008
468	The Simpsons Movie	Comedy	90	78	75	2007
484	The Twilight Saga: New Moon	Drama	27	78	50	2009
488	The Visitor	Drama	92	83	4	2007
492	There Will Be Blood	Drama	91	84	25	2007
499	Transformers	Action	57	89	150	2007
501	Transformers: Revenge of the Fallen	Action	20	76	210	2009
516	Up in the Air	Drama	90	76	25	2009
557	Zombieland	Action	90	87	24	2009

```
In [81]: df["AudienceRatings "]
```

```
Out[81]: 0      81
          1      44
          2      52
          3      84
          4      70
          ..
        554     36
        555     52
        556     73
        557     87
        558     42
        Name: AudienceRatings , Length: 559, dtype: int64
```

```
In [82]: df[df["Budget"]>200]
```

Out[82]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
33	Avatar	Action	83	92	237	2009
167	Harry Potter and the Half-Blood Prince	Adventure	83	75	250	2009
303	Pirates of the Caribbean: On Stranger Tides	Action	34	61	250	2011
304	Pirates of the Caribbean: At World's End	Action	45	74	300	2007
314	Quantum of Solace	Action	64	62	230	2008
360	Spider-Man 3	Action	61	54	258	2007
501	Transformers: Revenge of the Fallen	Action	20	76	210	2009

```
In [83]: df.drop("Genre",axis=1)
```

Out[83]:

	Film	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	87	81	8	2009
1	10,000 B.C.	9	44	105	2008
2	12 Rounds	30	52	20	2009
3	127 Hours	93	84	18	2010
4	17 Again	55	70	20	2009
...	...	...	...	...	...
554	Your Highness	26	36	50	2011
555	Youth in Revolt	68	52	18	2009
556	Zodiac	89	73	65	2007
557	Zombieland	90	87	24	2009
558	Zookeeper	14	42	80	2011

559 rows × 5 columns

```
In [84]: df.head()
```

Out[84]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

most of the pandas function returns new dataframe but original data frame is unchanged if we use inplace=True the changes are directly applied to the original data frame

```
In [85]: df.loc[df["Budget"]>200,"high_budget"]=True
```

```
In [86]: df
```

Out[86]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year	high_budget
0	(500) Days of Summer	Comedy	87	81	8	2009	NaN
1	10,000 B.C.	Adventure	9	44	105	2008	NaN
2	12 Rounds	Action	30	52	20	2009	NaN
3	127 Hours	Adventure	93	84	18	2010	NaN
4	17 Again	Comedy	55	70	20	2009	NaN
...	...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011	NaN
555	Youth in Revolt	Comedy	68	52	18	2009	NaN
556	Zodiac	Thriller	89	73	65	2007	NaN
557	Zombieland	Action	90	87	24	2009	NaN
558	Zookeeper	Comedy	14	42	80	2011	NaN

559 rows × 7 columns

In [87]:

df[20:50]

Out[87]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year	high_budget
20	Alien vs. Predator -- Requiem	Horror	14	37	40	2007	NaN
21	Aliens in the Attic	Adventure	30	46	45	2009	NaN
22	All About Steve	Comedy	6	35	15	2009	NaN
23	All Good Things	Drama	33	64	20	2010	NaN
24	Amelia	Adventure	21	35	40	2009	NaN
25	American Gangster	Thriller	79	87	100	2007	NaN
26	An Education	Drama	94	78	8	2009	NaN
27	Anonymous	Drama	46	66	30	2011	NaN
28	Apollo 18	Horror	23	31	5	2011	NaN
29	Appaloosa	Action	77	55	20	2008	NaN
30	Armored	Action	40	34	20	2009	NaN
31	Arthur	Comedy	26	49	40	2011	NaN
32	Australia	Adventure	55	69	78	2008	NaN
33	Avatar	Action	83	92	237	2009	True
34	Away We Go	Comedy	67	74	21	2009	NaN
35	Baby Mama	Comedy	63	59	70	2008	NaN
36	Babylon A.D.	Action	7	32	45	2008	NaN
37	Bad Teacher	Comedy	44	38	20	2011	NaN
38	Bandslam	Comedy	79	60	20	2009	NaN
39	Bangkok Dangerous	Action	9	33	45	2008	NaN
40	Battle: Los Angeles	Action	35	50	70	2011	NaN
41	Be Kind, Rewind	Comedy	65	57	20	2008	NaN
42	Beastly	Romance	19	50	17	2011	NaN
43	Bedtime Stories	Comedy	25	63	80	2008	NaN

	Film	Genre	CriticRating	AudienceRatings	Budget	Year	high_budget
44	Beginners	Comedy	84	80	4	2010	NaN
45	Beowulf	Action	71	52	150	2007	NaN
46	Beverly Hills Chihuahua	Adventure	41	56	20	2008	NaN
47	Black Swan	Drama	88	86	13	2010	NaN
48	Blades of Glory	Comedy	69	66	61	2007	NaN
49	Body of Lies	Action	53	65	68	2008	NaN

```
In [88]: df["lowbudget"]=df["Budget"]<200
df["lowbudget"]
```

Out[88]: 0 True
1 True
2 True
3 True
4 True
...
554 True
555 True
556 True
557 True
558 True
Name: lowbudget, Length: 559, dtype: bool

```
In [89]: df["lowbudget"]=df["Budget"].apply(lambda x:"Low" if x<100 else "high")
```

```
In [90]: df["lowbudget"]
```

Out[90]: 0 Low
1 high
2 Low
3 Low
4 Low
...
554 Low
555 Low
556 Low
557 Low
558 Low
Name: lowbudget, Length: 559, dtype: object

Adding new column based on calculations

```
In [91]: df["GenreRatings"]=df["Genre"] + "(" +df["AudienceRatings"].astype(str) + ")"
```

```
In [92]: df
```

Out[92]:

	Film	Genre	CriticRating	AudienceRatings	Budget	Year	high_budget	low_budget
--	------	-------	--------------	-----------------	--------	------	-------------	------------

0	(500) Days of Summer	Comedy	87	81	8	2009	NaN	NaN
1	10,000 B.C.	Adventure	9	44	105	2008	NaN	NaN
2	12 Rounds	Action	30	52	20	2009	NaN	NaN
3	127 Hours	Adventure	93	84	18	2010	NaN	NaN
4	17 Again	Comedy	55	70	20	2009	NaN	NaN
...	...	...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011	NaN	NaN
555	Youth in Revolt	Comedy	68	52	18	2009	NaN	NaN
556	Zodiac	Thriller	89	73	65	2007	NaN	NaN
557	Zombieland	Action	90	87	24	2009	NaN	NaN
558	Zookeeper	Comedy	14	42	80	2011	NaN	NaN

559 rows × 9 columns



```
In [93]: ##### Display options
pd.set_option('display.max_rows',10)
pd.set_option('display.max_columns',4)
```

```
In [94]: df
```

Out[94]:

	Film	Genre	...	lowbudget	GenreRatings
0	(500) Days of Summer	Comedy	...	Low	Comedy(81)
1	10,000 B.C.	Adventure	...	high	Adventure(44)
2	12 Rounds	Action	...	Low	Action(52)
3	127 Hours	Adventure	...	Low	Adventure(84)
4	17 Again	Comedy	...	Low	Comedy(70)
...	...	...	...	...	...
554	Your Highness	Comedy	...	Low	Comedy(36)
555	Youth in Revolt	Comedy	...	Low	Comedy(52)
556	Zodiac	Thriller	...	Low	Thriller(73)
557	Zombieland	Action	...	Low	Action(87)
558	Zookeeper	Comedy	...	Low	Comedy(42)

559 rows × 9 columns

Merge DataFrame

the merge function is used to join two dataframes based on their same column values

In [95]:

```
df1=pd.DataFrame({
    "Name":['Jeevan', 'Raavan', 'Geeta', 'Bheem'],
    'Age':[25, 24, 52, 40],
    'Qualification':['Msc', 'MA', 'MCA', 'Phd']})

df1
```

Out[95]:

	Name	Age	Qualification
0	Jeevan	25	Msc
1	Raavan	24	MA
2	Geeta	52	MCA
3	Bheem	40	Phd

In [96]:

```
df2=pd.DataFrame({'Name':['Jeevan', 'Raavan', 'Geeta', 'Bheem'],
    'Salary':[100000, 50000, 20000, 40000]})

df2
```

Out[96]:

	Name	Salary
0	Jeevan	100000
1	Raavan	50000
2	Geeta	20000
3	Bheem	40000

```
In [97]: dfm=pd.merge(df1,df2)
dfm
```

Out[97]:

	Name	Age	Qualification	Salary
0	Jeevan	25	Msc	100000
1	Raavan	24	MA	50000
2	Geeta	52	MCA	20000
3	Bheem	40	Phd	40000

```
In [98]: df
```

Out[98]:

	Film	Genre	...	lowbudget	GenreRatings
0	(500) Days of Summer	Comedy	...	Low	Comedy(81)
1	10,000 B.C.	Adventure	...	high	Adventure(44)
2	12 Rounds	Action	...	Low	Action(52)
3	127 Hours	Adventure	...	Low	Adventure(84)
4	17 Again	Comedy	...	Low	Comedy(70)
...	...	...	...	...	...
554	Your Highness	Comedy	...	Low	Comedy(36)
555	Youth in Revolt	Comedy	...	Low	Comedy(52)
556	Zodiac	Thriller	...	Low	Thriller(73)
557	Zombieland	Action	...	Low	Action(87)
558	Zookeeper	Comedy	...	Low	Comedy(42)

559 rows × 9 columns

```
In [99]: df
```

Out[99]:

	Film	Genre	...	lowbudget	GenreRatings
0	(500) Days of Summer	Comedy	...	Low	Comedy(81)
1	10,000 B.C.	Adventure	...	high	Adventure(44)
2	12 Rounds	Action	...	Low	Action(52)
3	127 Hours	Adventure	...	Low	Adventure(84)
4	17 Again	Comedy	...	Low	Comedy(70)
...	...	...	...	...	...
554	Your Highness	Comedy	...	Low	Comedy(36)
555	Youth in Revolt	Comedy	...	Low	Comedy(52)
556	Zodiac	Thriller	...	Low	Thriller(73)
557	Zombieland	Action	...	Low	Action(87)
558	Zookeeper	Comedy	...	Low	Comedy(42)

559 rows × 9 columns

In [100...
df.groupby("Genre")["Film"].count()

Out[100...
Genre
Action 154
Adventure 29
Comedy 172
Drama 101
Horror 49
Romance 18
Thriller 36
Name: Film, dtype: int64

In [101...
gen=df.groupby(["Genre", "Year"])["Budget"].sum()
print(gen)

Genre Year
Action 2007 1985
2008 2427
2009 2726
2010 3030
2011 2865
...
Romance 2011 435
Thriller 2007 795
2008 560
2010 120
2011 493
Name: Budget, Length: 33, dtype: int64